Research Methodology

PCNME: Predictive Cloud-Native Mobile Edge

A Mathematical Framework for Optimized Task Offloading in IoT–Fog–Cloud Systems

---

Author: Imed Eddine Benmadi

Keywords: Task offloading, NSGA-II, MMDE, Execution Cost, Pareto optimisation, deep reinforcement learning, DAG, proactive handoff, vehicular fog computing

---

April 2026

eötvös loránd university - Data Science

Abstract

This paper presents the complete mathematical methodology of PCNME, the Predictive Cloud-Native Mobile Edge framework for intelligent task offloading in three-tier IoT–Fog–Cloud systems. The problem of routing computation-intensive tasks from resource-constrained IoT vehicles to fog or cloud servers is formulated as a multi-objective combinatorial optimization problem with deadline and capacity constraints. We define all system variables and derive formal expressions for ***execution cost, task latency, and energy consumption***.

The methodology integrates five algorithmic components: an Execution Cost classifier that pre-filters tasks before optimization, an MMDE-enhanced NSGA-II multi-objective optimizer that produces Pareto-optimal offline routing plans, a behavioral cloning pipeline that seeds a Deep Q-Network agent from those plans, an online DQN agent that adapts placement decisions to live network conditions, and a GPS-driven proactive handoff predictor that prevents task failures when vehicles cross fog coverage boundaries. Every variable, equation, and parameter is defined formally and its physical interpretation is stated. The objective of this methodology section is to provide a complete, self-contained mathematical foundation from which an implementation can be derived without ambiguity.

Contents

1 Parameters **3**

2 System Model and Notation **5**
2.1 Network Topology 5
2.2 Vehicle Model 6
2.3 DAG Task Model 6
2.4 Offloading Decision Variable 7

3 Latency and Energy Models **7**
3.1 Transmission Time 7
3.2 Execution Time on Fog 7
3.3 Execution Time on Cloud 7
3.4 Total Step Latency 8
3.5 Energy Model 8

4 Multi-Objective Optimisation Problem **8**
4.1 Problem Formulation 8

5 Execution Cost Classification **9**
5.1 Execution Cost Metric 9
5.2 Boulder and Pebble Classification 9
5.3 Aggregator and Avalanche Prevention 9

6 MMDE-Enhanced NSGA-II **10**
6.1 Chromosome Representation . . . 10
6.2 Objective Evaluation . . . 10
6.3 Non-Dominated Sorting . . . 10
6.4 Crowding Distance . . . 11
6.5 Minimax Differential Evolution Mutation . . . 11
6.6 Knee Point Selection . . . 12

7 Deep Q-Network Task Placement Agent **12**
7.1 Markov Decision Process Formulation . . . 12
7.2 State Representation . . . 13
7.3 Q-Network Architecture . . . 13
7.4 Reward Function . . . 14
7.5 Experience Replay and Temporal Difference Learning . . . 14
7.6 Behavioral Cloning Pre-training . . . 15
7.7 Offline Training Data Generation . . . 15
7.8 Offline-to-Online Transition . . . 15

8 Proactive Mobility Management **16**
8.1 Zone Exit Time Estimation . . . 16
8.2 Handoff Mode Selection . . . 16
8.3 Reactive Fallback: NTB/HTB System . . . 17
8.4 Step-Level Assessment for DAG Tasks . . . 17

9 Computational Complexity Analysis **18**
9.1 Offline Phase: TOF-MMDE-NSGA-II . . . 18
9.2 Online Phase: DQN Inference . . . 18
9.3 Mobility Prediction . . . 18
9.4 Summary . . . 19

10 End-to-End Processing Pipeline **19**

11 Evaluation Metrics **20**

1 Parameters

Table 1 lists all parameters of the PCNME framework with their symbols, default values, and justifications. Sensitivity analysis over the key parameters θ, F, CR, γ, and ω_L is recommended as part of the evaluation.

Table 1 collects every parameter with its symbol as used in the mathematical formulation, its environment variable name, its default value, and the source or physical justification for that default. The table is organised by subsystem to match the structure of the methodology.

Table 1: Complete parameter set of the PCNME framework.

Symbol	**Description**	**Env variable**	**Default**	**Source / Justification**
Compute and network				
μ_k	Fog node MIPS capacity	`FOG_MIPS`	2,000	Edge server benchmark
μ_c	Cloud MIPS capacity	`CLOUD_MIPS`	8,000	4× fog (GPU server)
B	IoT upload bandwidth (Mbps)	`BANDWIDTH_MBPS`	100	5G NR sub-6 GHz
B_c	Fog–cloud backbone (Mbps)	`FOG_CLOUD_BW`	1,000	Urban fibre link
B_{ff}	Fog–fog mesh (Mbps)	`FOG_FOG_BW`	100	Inter-node mesh
δ_{5G}	5G one-way latency (ms)	`G5_LATENCY_MS`	2	3GPP Release 16
δ_{WAN}	Fog–cloud WAN delay (ms)	`WAN_LATENCY_MS`	30	Istanbul–cloud typical
α	WAN energy penalty factor	`WAN_ENERGY_ALPHA`	1.8	Longer path coefficient
κ	Fog compute energy (J/MI)	`COMPUTE_ENERGY_KAPPA`	0.001	Calibrated to ARM server
P_{tx}	Device transmit power (W)	`TX_POWER_WATTS`	0.5	5G UE typical
TOF-Broker				
θ	EC classification threshold (s)	`EC_THRESHOLD`	1.0	Nouri et al. [2]
$Q_{\max}$	Aggregator trigger depth	`Q_MAX`	50	Empirically tuned
NSGA-II + MMDE				
n	Population size	`NSGA_POP_SIZE`	100	Standard for this scale

Table 1 continued

Symbol	Description	Env variable	Default	Source / Justification
G	Number of generations	`NSGA_GENS`	200	Convergence criterion
F	MMDE scaling factor	`MMDE_F`	0.5	Jafari & Rezvani [1]
CR	MMDE crossover rate	`MMDE_CR`	0.9	Jafari & Rezvani [1]
P_{batch}	Tasks per offline batch	`NSGA_BATCH_SIZE`	100	Memory–compute trade-off
N_{batches}	Offline batches	`N_OFFLINE_BATCHES`	1,000	Training set size
DQN Agent				
$\lvert\mathbf{s}\rvert$	State dimension	`AGENT1_STATE_DIM`	13	Eq. (21)
$\lvert\mathcal{A}\rvert$	Action dimension	`AGENT1_ACTION_DIM`	5	$K+1$ destinations
$\boldsymbol{h}$	Hidden layer widths	`AGENT1_HIDDEN`	[256, 128]	Architecture search
η	Adam learning rate	`AGENT1_LR`	0.001	Kingma & Ba (2015)
γ	Discount factor	`AGENT1_GAMMA`	0.95	Near-future focused
ε_0	Initial exploration rate	`AGENT1_EPSILON_START`	0.30	Standard DQN setting
$\varepsilon_{\min}$	Minimum exploration rate	`AGENT1_EPSILON_END`	0.05	Maintain exploration
T_ε	Epsilon decay horizon (steps)	`AGENT1_EPSILON_DECAY`	10,000	10 min at 10 tasks/s
B	Mini-batch size	`AGENT1_BATCH_SIZE`	64	GPU memory trade-off
$\lvert\mathcal{B}\rvert_{\max}$	Replay buffer size	`AGENT1_BUFFER_SIZE`	50,000	Decorrelation coverage
T_{sync}	Target network sync (steps)	`AGENT1_TARGET_UPDATE`	1,000	Stable target period
δ	Huber loss threshold	`HUBER_DELTA`	1.0	Standard DQN value
Reward weights				
ω_L	Latency weight	`REWARD_WEIGHT_LATENCY`	0.5	Primary objective
ω_E	Energy weight	`REWARD_WEIGHT_ENERGY`	0.3	Secondary objective
ω_V	Violation weight	`REWARD_WEIGHT_VIOLATION`	0.2	Constraint enforcement
λ_{crit}	Safety multiplier	`DEADLINE_PENALTY_CRITICAL`	10.0	Safety-critical penalty
E_{ref}	Energy normalisation ref. (J)	`ENERGY_REF_JOULES`	0.10	Typical pebble energy
Mobility and handoff				

Table 1 continued

Symbol	Description	Env variable	Default	Source / Justification
R_k	Fog coverage radius (m)	FOG_COVERAGE_RADIUS	250	5G small cell radius
$T_{\max}$	T_{exit} normalisation bound (s)	T_EXIT_MAX_S	10.0	Max useful lookahead
$s_{\max}$	Speed normalisation bound (m/s)	SPEED_MAX_MS	33.3	120 km/h maximum
Simulation				
N	Number of vehicles	N_VEHICLES	50	Urban density scenario
λ_{task}	Task arrival rate per vehicle (Hz)	TASK_RATE_HZ	10	10 fps camera
T_{sim}	Simulation duration (s)	SIM_DURATION_S	600	10-minute evaluation
T_{warm}	Warm-up period (s)	WARMUP_S	60	Agent stabilisation
—	Random seed	RANDOM_SEED	42	Reproducibility
—	Independent runs	N_RUNS	5	Statistical validity

2 System Model and Notation

2.1 Network Topology

The system operates over a three-tier hierarchy. Let $\mathcal{F} = \{f_1, f_2, \ldots, f_K\}$ be the set of K fog nodes and $\mathcal{C}$ denote the cloud server. The full set of compute destinations is:

$$\mathcal{X} = \mathcal{F} \cup \{\mathcal{C}\} \tag{1}$$

Each fog node $f_k \in \mathcal{F}$ is characterised by a tuple:

$$f_k = \langle \boldsymbol{p}_k,\ R_k,\ \mu_k,\ \rho_k(t) \rangle \tag{2}$$

where $\boldsymbol{p}_k \in \mathbb{R}^2$ is the fog's two-dimensional position in metres, R_k is its coverage radius in metres, μ_k is its compute capacity in Million Instructions Per Second (MIPS), and $\rho_k(t) \in [0, 1]$ is its CPU utilisation at time t, which evolves dynamically as tasks arrive and depart.

The cloud is characterised by:

$$\mathcal{C} = \langle \mu_c,\ \delta_{\text{WAN}} \rangle \tag{3}$$

where $\mu_c > \mu_k$ is the cloud's compute capacity and δ_{WAN} is the fixed wide-area network propagation delay in milliseconds.

Static network assumption. The network infrastructure — link bandwidths, propagation delays, and fog node positions — is assumed *static* throughout the system's

operational lifetime. Specifically, the upload bandwidth B, the fog-to-cloud backbone bandwidth B_c, and the WAN delay δ_{WAN} are fixed constants. This assumption reflects the real-world constraint that a deployed fog infrastructure does not dynamically reconfigure its physical links. Only the per-node CPU utilisation $\rho_k(t)$ and queue depth $q_k(t)$ vary at runtime, driven by the arrival and departure of tasks. As a consequence, network resource allocation is not an optimisation variable in this framework; the optimisation concerns exclusively the *compute placement* decision $\mathbf{x}$ — which node executes each task step.

2.2 Vehicle Model

Let $\mathcal{V} = \{v_1, v_2, \ldots, v_N\}$ be a fleet of N mobile IoT vehicles. At any time t, vehicle v_i is described by its *Spatial Tag*:

$$\sigma_i(t) = \langle \boldsymbol{q}_i(t),\ s_i(t),\ \phi_i(t) \rangle \tag{4}$$

where $\boldsymbol{q}_i(t) \in \mathbb{R}^2$ is the vehicle's GPS-derived position, $s_i(t)$ is its speed in metres per second, and $\phi_i(t) \in [0, 2\pi)$ is its heading in radians measured from the positive x-axis. The Cartesian velocity vector of vehicle v_i is:

$$\boldsymbol{u}_i(t) = s_i(t) \begin{pmatrix} \cos\phi_i(t) \\ \sin\phi_i(t) \end{pmatrix} \tag{5}$$

Vehicle v_i is said to be *in coverage* of fog node f_k at time t if:

$$\|\boldsymbol{q}_i(t) - \boldsymbol{p}_k\|_2 \leq R_k \tag{6}$$

2.3 DAG Task Model

Following the formalization in Zhu et al. [4], each vehicle generates application workloads modeled as Directed Acyclic Graphs. A task τ submitted by vehicle v_i at time t_0 is defined as:

$$\tau = \langle G,\ D_\tau,\ v_i,\ t_0 \rangle \tag{7}$$

where $G = (\mathcal{V}_G, \mathcal{E}_G)$ is the DAG, D_τ is the end-to-end deadline in milliseconds, and v_i is the originating vehicle.

Each node $g_j \in \mathcal{V}_G$ represents one processing step with properties:

$$g_j = \langle \ell_j,\ d_j^{\text{in}},\ d_j^{\text{out}},\ d_j^{\text{local}} \rangle \tag{8}$$

where ℓ_j is computational demand in MI, d_j^{in} and d_j^{out} are input and output data sizes in kilobytes, and d_j^{local} is the per-step deadline in milliseconds.

A directed edge $(g_j, g_m) \in \mathcal{E}_G$ means step g_m cannot begin execution until step g_j has completed and transmitted its output. When two consecutive steps are co-located on the same node, the inter-step transfer time is zero.

Definition 2.1 (Critical Path)**.** The *critical path* $\Pi \subseteq \mathcal{V}_G$ is the path from the source step to the sink step that maximises total execution time.

End-to-end application latency is:

$$L_\tau(\mathbf{x}) = \sum_{g_j \in \Pi} \Big[T_{\text{exec}}(g_j, x_j) + T_{\text{tx}}(g_j, g_{j+1}, x_j, x_{j+1}) \Big] \tag{9}$$

where $x_j \in \mathcal{X}$ is the destination assigned to step g_j, and $T_{\text{tx}} = 0$ when $x_j = x_{j+1}$.

2.4 Offloading Decision Variable

The offloading decision for a task τ with M offloadable steps is a vector:

$$\mathbf{x} = (x_1, x_2, \ldots, x_M), \quad x_j \in \mathcal{X} \tag{10}$$

Step g_1 is always executed on the originating device (it handles local capture and compression) and is therefore excluded from $\mathbf{x}$.

3 Latency and Energy Models

3.1 Transmission Time

The time to transmit d kilobytes from the IoT device to a fog node over a channel with nominal bandwidth B Mbps is:

$$T_{\text{tx}}^{\text{up}}(d, B) = \frac{d \times 8}{B \times 10^3} \times 10^3 = \frac{8d}{B} \quad [\text{ms}] \tag{11}$$

where the factor of 10^3 converts seconds to milliseconds. A fixed 5G access latency δ_{5G} (one-way, typically 1–3 ms) is added:

$$T_{\text{access}}(d) = T_{\text{tx}}^{\text{up}}(d, B) + \delta_{5G} \tag{12}$$

For fog-to-cloud transmission with backbone bandwidth B_c and WAN propagation delay δ_{WAN}:

$$T_{\text{tx}}^{\text{cloud}}(d) = \frac{8d}{B_c} + \delta_{\text{WAN}} \quad [\text{ms}] \tag{13}$$

3.2 Execution Time on Fog

The nominal execution time of step g_j on fog node f_k at full capacity is:

$$T_{\text{exec}}^{0}(g_j, f_k) = \frac{\ell_j}{\mu_k} \times 10^3 \quad [\text{ms}] \tag{14}$$

Under non-zero load $\rho_k(t)$, the effective execution time accounting for CPU time-sharing is:

$$T_{\text{exec}}(g_j, f_k, t) = \frac{\ell_j}{\mu_k\,(1 - \rho_k(t))} \times 10^3 \quad [\text{ms}] \tag{15}$$

Remark 3.1. Equation (15) assumes a work-conserving processor sharing model. This is consistent with the model adopted in Jafari and Rezvani [1] and Nouri et al. [2]. The divisor $(1 - \rho_k(t))$ degrades toward zero as load approaches unity, capturing queue starvation.

3.3 Execution Time on Cloud

Cloud execution uses dedicated GPU-accelerated servers with fixed capacity μ_c:

$$T_{\text{exec}}(g_j, \mathcal{C}) = \frac{\ell_j}{\mu_c} \times 10^3 \quad [\text{ms}] \tag{16}$$

Cloud load is assumed manageable due to elastic scaling; no load penalty is applied.

3.4 Total Step Latency

The total latency incurred by assigning step g_j to destination x_j is:

$$L_j(x_j, t) = \begin{cases} T_{\text{access}}(d_j^{\text{in}}) + T_{\text{exec}}(g_j, x_j, t) & \text{if } x_j \in \mathcal{F} \\ T_{\text{tx}}^{\text{cloud}}(d_j^{\text{in}}) + T_{\text{exec}}(g_j, \mathcal{C}) & \text{if } x_j = \mathcal{C} \end{cases} \tag{17}$$

3.5 Energy Model

Transmission energy for uploading d kilobytes at transmit power P_{tx} watts over a link occupying T_{tx} milliseconds is:

$$E_{\text{tx}}(d) = P_{\text{tx}} \cdot \frac{8d}{B \times 10^3} \quad \text{[Joules]} \tag{18}$$

Fog computation energy for step g_j, assuming a compute energy coefficient κ (Joules per Million Instructions):

$$E_{\text{comp}}(g_j) = \kappa \cdot \ell_j \quad \text{[Joules]} \tag{19}$$

Total energy cost for assigning step g_j to x_j:

$$E_j(x_j) = E_{\text{tx}}(d_j^{\text{in}}) + \begin{cases} E_{\text{comp}}(g_j) & \text{if } x_j \in \mathcal{F} \\ \alpha \cdot E_{\text{tx}}(d_j^{\text{in}}) & \text{if } x_j = \mathcal{C} \end{cases} \tag{20}$$

where $\alpha > 1$ is a WAN energy penalty factor reflecting the longer transmission distance to the cloud data centre.

4 Multi-Objective Optimisation Problem

4.1 Problem Formulation

Let $\mathcal{T} = \{\tau_1, \tau_2, \ldots, \tau_N\}$ be a batch of N tasks, each with M offloadable steps, giving a joint decision vector $\mathbf{X} \in \mathcal{X}^{N \times M}$. The offloading problem is formulated as:

$$\min_{\mathbf{X}} \quad \boldsymbol{f}(\mathbf{X}) = \begin{pmatrix} f_1(\mathbf{X}) \\ f_2(\mathbf{X}) \end{pmatrix} = \begin{pmatrix} \displaystyle\sum_{i=1}^{N} \sum_{j=1}^{M} E_j(x_{ij}) \\ \displaystyle\sum_{i=1}^{N} L_{\tau_i}(\mathbf{x}_i) \end{pmatrix} \tag{21}$$

subject to:

$$L_{\tau_i}(\mathbf{x}_i) \leq D_{\tau_i} \quad \forall\, i \in \{1, \ldots, N\} \tag{22}$$

$$\sum_{i,j:\, x_{ij} = f_k} \ell_{ij} \leq C_k \quad \forall\, f_k \in \mathcal{F} \tag{23}$$

$$x_{ij} \in \mathcal{X} \quad \forall\, i, j \tag{24}$$

Constraint (22) enforces per-task end-to-end deadline satisfaction. Constraint (23) bounds aggregate MI load per fog node by its effective capacity $C_k = \mu_k \cdot (1 - \rho_k^{\min})$, where $\rho_k^{\min}$ is a minimum reserved utilisation margin.

Proposition 4.1. Problem (21)–(24) is NP-hard. The number of feasible solutions is $|\mathcal{X}|^{N \times M}$, which grows exponentially with both the number of tasks and the number of DAG steps. Exact optimisation is computationally intractable at IoT deployment scale.

Definition 4.1 (Pareto Dominance)**.** Solution $\mathbf{X}^{(a)}$ *Pareto-dominates* $\mathbf{X}^{(b)}$, written $\mathbf{X}^{(a)} \prec \mathbf{X}^{(b)}$, if and only if:

$$f_k(\mathbf{X}^{(a)}) \leq f_k(\mathbf{X}^{(b)}) \;\; \forall k \in \{1, 2\}, \quad \text{and} \quad f_j(\mathbf{X}^{(a)}) < f_j(\mathbf{X}^{(b)}) \;\; \text{for some } j \tag{25}$$

Definition 4.2 (Pareto Front)**.** The *Pareto front* $\mathcal{P}^* \subset \mathcal{X}^{N \times M}$ is the set of all solutions that are not dominated by any other feasible solution:

$$\mathcal{P}^* = \left\{ \mathbf{X} \mid \nexists\, \mathbf{X}' : \mathbf{X}' \prec \mathbf{X} \right\} \tag{26}$$

No element of $\mathcal{P}^*$ can improve on f_1 without degrading f_2, and vice versa.

5 Execution Cost Classification

5.1 Execution Cost Metric

The Execution Cost (EC) of a step g_j, introduced by Nouri et al. [2], is the time the step would occupy a fog node running at full capacity:

$$\mathrm{EC}(g_j) = \frac{\ell_j}{\mu_k} \quad \text{[seconds]} \tag{27}$$

Since μ_k is uniform across fog nodes in this deployment, EC is task-dependent only and can be computed without knowledge of which fog node will be assigned.

5.2 Boulder and Pebble Classification

Let $\theta > 0$ be the EC threshold parameter. Step g_j is classified as:

$$\mathrm{class}(g_j) = \begin{cases} \textsc{Boulder} & \text{if } \mathrm{EC}(g_j) \geq \theta \\ \textsc{Pebble} & \text{if } \mathrm{EC}(g_j) < \theta \end{cases} \tag{28}$$

Boulders are immediately routed to cloud without entering the optimisation pipeline. Pebbles are queued for multi-objective optimisation. This partitioning reduces the search space of problem (21) by excluding steps that cannot feasibly complete on any fog node within the deadline.

5.3 Aggregator and Avalanche Prevention

Let $Q(t)$ denote the pebble queue depth at time t and $Q_{\max}$ a configurable overflow threshold. The *Aggregator* activates when:

$$Q(t) > Q_{\max} \tag{29}$$

Upon activation, all queued pebbles are bundled into a *super-task* $\mathcal{S}$ and routed to cloud:

$$\mathcal{S} = \left\{ g_j \in Q(t) \;\middle|\; Q(t) > Q_{\max} \right\} \tag{30}$$

This prevents queue cascade failure (the Pebble Avalanche) identified as an open gap in the literature [2]. The total latency of a super-task is bounded by:

$$L_{\mathcal{S}} \leq \frac{8 \cdot \sum_{g_j \in \mathcal{S}} d_j^{\text{in}}}{B_c} + \delta_{\text{WAN}} + \max_{g_j \in \mathcal{S}} T_{\text{exec}}(g_j, \mathcal{C}) \tag{31}$$

assuming parallel cloud execution of all bundled steps.

6 MMDE-Enhanced NSGA-II

6.1 Chromosome Representation

A candidate solution (chromosome) for a batch of P pebble steps is a vector of integer genes:

$$\mathbf{c} = (c_1, c_2, \ldots, c_P), \quad c_j \in \{0, 1, \ldots, |\mathcal{F}|\} \tag{32}$$

where gene value $c_j = k$ assigns step g_j to fog node f_k, and $c_j = |\mathcal{F}|$ assigns it to the cloud. The population at generation r is $\mathbf{C}^{(r)} = \{\mathbf{c}_1^{(r)}, \ldots, \mathbf{c}_n^{(r)}\}$ where $n = |\mathbf{C}|$ is the population size.

6.2 Objective Evaluation

Each chromosome $\mathbf{c}$ is evaluated on both objectives:

$$F_1(\mathbf{c}) = \sum_{j=1}^{P} E_j(x_j(\mathbf{c})) \tag{33}$$

$$F_2(\mathbf{c}) = \sum_{j=1}^{P} L_j(x_j(\mathbf{c}), t) \tag{34}$$

where $x_j(\mathbf{c})$ maps gene c_j to its corresponding destination in $\mathcal{X}$.

6.3 Non-Dominated Sorting

NSGA-II partitions the population into Pareto fronts $\mathcal{F}_1, \mathcal{F}_2, \ldots$ via non-dominated sorting [1]. For each chromosome $\mathbf{c}_i^{(r)}$, its domination count n_i (number of solutions that dominate it) and its dominated set S_i (solutions it dominates) are computed. Front $\mathcal{F}_1$ contains all solutions with $n_i = 0$. Subsequent fronts are obtained by successively removing $\mathcal{F}_1$ and recomputing.

6.4 Crowding Distance

Within each front, crowding distance d_i^{crowd} measures the perimeter of the cuboid formed by the nearest neighbours of solution i in the objective space. For two objectives:

$$d_i^{\text{crowd}} = \sum_{k=1}^{2} \frac{F_k(\mathbf{c}_{i+1}) - F_k(\mathbf{c}_{i-1})}{F_k^{\max} - F_k^{\min}} \tag{35}$$

where solutions are sorted by objective k and boundary solutions receive $d_i^{\text{crowd}} = \infty$. Larger crowding distance is preferred when two solutions share the same front rank, ensuring diversity preservation.

6.5 Minimax Differential Evolution Mutation

Standard NSGA-II uses polynomial mutation, which samples perturbations uniformly and wastes evaluations on unproductive regions. The Minimax Differential Evolution (MMDE) operator, proposed in [1], replaces this with a directional mutation derived from population structure.

Given three distinct solutions $\mathbf{r}_1, \mathbf{r}_2, \mathbf{r}_3 \in \mathbf{C}^{(r)}$, the mutant vector is:

$$\mathbf{V} = \mathbf{r}_1 + F \cdot (\mathbf{r}_2 - \mathbf{r}_3) \tag{36}$$

where $F \in (0, 2)$ is the differential scaling factor. For integer gene representations, the mutation of gene j is:

$$V_j = \text{clip}\big(r_{1,j} + \lfloor F \cdot (r_{2,j} - r_{3,j}) + 0.5 \rfloor,\ 0,\ |\mathcal{X}| - 1\big) \tag{37}$$

Binomial crossover then combines the mutant with the target chromosome $\mathbf{c}_i^{(r)}$ to produce a trial chromosome $\mathbf{u}_i$:

$$u_{i,j} = \begin{cases} V_j & \text{if } \xi_j \leq \text{CR or } j = j_{\text{rand}} \\ c_{i,j}^{(r)} & \text{otherwise} \end{cases} \tag{38}$$

where $\xi_j \sim \mathcal{U}(0, 1)$ is drawn independently per gene, $\text{CR} \in (0, 1)$ is the crossover rate, and j_{rand} is a randomly chosen index ensuring at least one gene is taken from the mutant.

The trial chromosome $\mathbf{u}_i$ replaces $\mathbf{c}_i^{(r)}$ in the next generation if it is not dominated:

$$\mathbf{c}_i^{(r+1)} = \begin{cases} \mathbf{u}_i & \text{if } \mathbf{u}_i \prec \mathbf{c}_i^{(r)} \text{ or } \mathbf{u}_i = \mathbf{c}_i^{(r)} \\ \mathbf{c}_i^{(r)} & \text{otherwise} \end{cases} \tag{39}$$

The full MMDE-NSGA-II algorithm is summarised in Algorithm 1.

Algorithm 1 TOF-MMDE-NSGA-II Offline Optimiser

Input: pebble step set $\mathcal{G}_P$, fog state $\{\rho_k\}$, population size n, generations G, scaling factor F, crossover rate CR
Output: Pareto front $\mathcal{P}^*$, knee point $\mathbf{c}^*$
Initialise population $\mathbf{C}^{(0)}$ with n random chromosomes
Evaluate $\boldsymbol{F}(\mathbf{c})$ for all $\mathbf{c} \in \mathbf{C}^{(0)}$
Compute non-dominated fronts and crowding distances
for $r = 1$ **to** G **do**
  for each $\mathbf{c}_i^{(r-1)} \in \mathbf{C}^{(r-1)}$ **do**
    Sample $\mathbf{r}_1, \mathbf{r}_2, \mathbf{r}_3$ from $\mathbf{C}^{(r-1)}$, all distinct
    Compute mutant $\mathbf{V}$ via Eq. (37)
    Compute trial $\mathbf{u}_i$ via Eq. (38)
    Evaluate $\boldsymbol{F}(\mathbf{u}_i)$
    Select $\mathbf{c}_i^{(r)}$ via Eq. (39)
  end for
  Combine $\mathbf{C}^{(r)} \cup \{\text{offspring}\}$, re-sort, select best n
end for
Extract Pareto front $\mathcal{P}^* = \mathcal{F}_1$ from final population
Compute knee point $\mathbf{c}^*$ via Eq. (40)
return $\mathcal{P}^*, \mathbf{c}^*$

6.6 Knee Point Selection

Let $\boldsymbol{F}^{\text{utopia}} = (F_1^{\min}, F_2^{\min})$ be the utopia point, where each component is the minimum of that objective over the Pareto front (achievable only if the objectives were independent). The knee point $\mathbf{c}^*$ is the Pareto-front solution with minimum normalised Euclidean distance to the utopia point:

$$\mathbf{c}^* = \arg\min_{\mathbf{c} \in \mathcal{P}^*} \left\| \begin{pmatrix} \dfrac{F_1(\mathbf{c}) - F_1^{\min}}{F_1^{\max} - F_1^{\min}} \\ \dfrac{F_2(\mathbf{c}) - F_2^{\min}}{F_2^{\max} - F_2^{\min}} \end{pmatrix} \right\|_2 \tag{40}$$

The knee point provides the best balance between energy minimisation and latency minimisation without requiring any preference weights to be specified a priori.

7 Deep Q-Network Task Placement Agent

7.1 Markov Decision Process Formulation

The real-time task placement problem is modeled as a Markov Decision Process (MDP) $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{R}, \mathcal{P}, \gamma)$, where:

- $\mathcal{S}$ is the state space — the agent's observable view of the system at the moment a pebble step arrives for placement.
- $\mathcal{A} = \mathcal{X} = \{f_1, f_2, \ldots, f_K, \mathcal{C}\}$ is the action space — the destination to which the step is assigned.

- $\mathcal{R} : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is the reward function.
- $\mathcal{P}(s' \mid s, a)$ is the transition probability — here implicit in the simulation dynamics.
- $\gamma \in (0, 1)$ is the discount factor.

7.2 State Representation

Under the static network assumption, the upload bandwidth B and all link capacities are fixed constants known at design time. The agent therefore does not need to observe network utilisation as a runtime state variable. The state vector $\mathbf{s} \in \mathbb{R}^{11}$ observed by the agent at decision time t for step g_j submitted by vehicle v_i is:

$$\mathbf{s} = \big(\rho_1(t),\ \rho_2(t),\ \rho_3(t),\ \rho_4(t),\ q_1(t),\ q_2(t),\ q_3(t),\ q_4(t),\ \widehat{\mathrm{EC}}_j,\ \widehat{s}_i(t),\ \widehat{T}_{\mathrm{exit}}\big) \tag{41}$$

where hats denote normalisation to $[0, 1]$. The components are:

- $\rho_k(t) \in [0, 1]$: CPU utilisation of fog node f_k ($k = 1, \ldots, 4$). This is the primary signal for load-aware placement.
- $q_k(t)/q_{\max}$: queue depth of fog node f_k normalised by the maximum queue depth $q_{\max}$. Captures queuing latency not reflected in CPU utilisation alone.
- $\widehat{\mathrm{EC}}_j = \min(\mathrm{EC}(g_j)/\theta, 1)$: normalised Execution Cost of the current step. Heavier steps require lighter-loaded nodes.
- $\widehat{s}_i(t) = s_i(t)/s_{\max}$: vehicle speed normalised by $s_{\max}$. Faster vehicles exit fog zones sooner, affecting placement urgency.
- $\widehat{T}_{\mathrm{exit}} = \min(T_{\mathrm{exit}}/T_{\max}, 1)$: normalised time until the vehicle exits the current fog zone. The core proactive handoff signal.

Remark 7.1. Two dimensions present in earlier formulations — upload link utilisation $\widehat{B}(t)$ and remaining deadline fraction $\widehat{d}_j^{\mathrm{rem}}$ — are removed. Under the static network assumption, B is a fixed constant and contributes no information for placement decisions. The deadline fraction is implicitly captured through the T_{exit} dimension: if a vehicle is about to leave the zone, the effective deadline for fog completion is already constrained. Reducing the state dimension from 13 to 11 shrinks the input weight matrix $\boldsymbol{W}^{(1)}$ from $\mathbb{R}^{256\times 13}$ to $\mathbb{R}^{256\times 11}$, marginally improving training efficiency without sacrificing decision quality.

7.3 Q-Network Architecture

The action-value function $Q(s, a; \boldsymbol{\theta})$ is approximated by a fully-connected neural network with parameters $\boldsymbol{\theta}$:

$$\boldsymbol{h}^{(1)} = \mathrm{ReLU}\big(\boldsymbol{W}^{(1)}\mathbf{s} + \boldsymbol{b}^{(1)}\big), \quad \boldsymbol{W}^{(1)} \in \mathbb{R}^{256\times 13} \tag{42}$$

$$\boldsymbol{h}^{(2)} = \mathrm{ReLU}\big(\boldsymbol{W}^{(2)}\boldsymbol{h}^{(1)} + \boldsymbol{b}^{(2)}\big), \quad \boldsymbol{W}^{(2)} \in \mathbb{R}^{128\times 256} \tag{43}$$

$$\boldsymbol{Q}(s; \boldsymbol{\theta}) = \boldsymbol{W}^{(3)}\boldsymbol{h}^{(2)} + \boldsymbol{b}^{(3)}, \quad \boldsymbol{W}^{(3)} \in \mathbb{R}^{|\mathcal{A}|\times 128} \tag{44}$$

The agent selects actions via ε-greedy policy:

$$a_t = \begin{cases} \text{random } a \in \mathcal{A} & \text{with probability } \varepsilon_t \\ \arg\max_{a\in\mathcal{A}} Q(s_t, a; \boldsymbol{\theta}) & \text{otherwise} \end{cases} \tag{45}$$

Exploration rate ε_t decays linearly:

$$\varepsilon_t = \max\left(\varepsilon_{\min},\ \varepsilon_0 - \frac{(\varepsilon_0 - \varepsilon_{\min})}{T_\varepsilon} \cdot t\right) \tag{46}$$

where ε_0 is the initial exploration rate, $\varepsilon_{\min}$ is the final minimum, and T_ε is the decay horizon in steps.

7.4 Reward Function

The reward received after step g_j completes with latency L_j, energy E_j, and deadline d_j^{local} is:

$$R(g_j, x_j) = -\omega_L \cdot \widetilde{L}_j - \omega_E \cdot \widetilde{E}_j - \omega_V \cdot \mathbf{1}[L_j > d_j^{\text{local}}] \cdot \lambda \tag{47}$$

where:

$$\widetilde{L}_j = \min\left(\frac{L_j}{d_j^{\text{local}}},\ 3\right) \tag{48}$$

$$\widetilde{E}_j = \min\left(\frac{E_j}{E_{\text{ref}}},\ 3\right) \tag{49}$$

$$\mathbf{1}[\cdot] \in \{0, 1\} \text{ is the deadline violation indicator} \tag{50}$$

$$\lambda \in \{1, \lambda_{\text{crit}}\} \text{ is the safety multiplier} \tag{51}$$

$\omega_L, \omega_E, \omega_V > 0$ are reward weights satisfying $\omega_L + \omega_E + \omega_V = 1$. $\lambda_{\text{crit}} \gg 1$ applies to safety-critical steps such as collision avoidance alerts, ensuring the agent prioritises their deadlines.

7.5 Experience Replay and Temporal Difference Learning

Transitions $(s_t, a_t, r_t, s_{t+1}, \text{done}_t)$ are stored in a replay buffer $\mathcal{B}$ of capacity $|\mathcal{B}|_{\max}$. At each update step, a mini-batch $\mathcal{B}' \subset \mathcal{B}$ of size B is sampled uniformly. The TD target for each transition in $\mathcal{B}'$ is computed using a target network $\boldsymbol{\theta}^-$, which is a periodically-frozen copy of the online network:

$$y_t = r_t + \gamma \cdot \max_{a' \in \mathcal{A}} Q(s_{t+1}, a'; \boldsymbol{\theta}^-) \cdot (1 - \text{done}_t) \tag{52}$$

The online network is updated by minimising the Huber loss over the mini-batch:

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{B} \sum_{(s,a,r,s') \in \mathcal{B}'} \mathcal{H}_\delta(y - Q(s, a; \boldsymbol{\theta})) \tag{53}$$

$$\mathcal{H}_\delta(u) = \begin{cases} \frac{1}{2}u^2 & \text{if } |u| \le \delta \\ \delta(|u| - \frac{1}{2}\delta) & \text{otherwise} \end{cases} \tag{54}$$

Gradients are computed by back-propagation and applied via Adam optimiser with learning rate η. The target network is synchronised every T_{sync} update steps:

$$\boldsymbol{\theta}^- \leftarrow \boldsymbol{\theta} \quad \text{every } T_{\text{sync}} \text{ steps} \tag{55}$$

7.6 Behavioral Cloning Pre-training

To eliminate cold-start degradation, the agent is pre-trained from NSGA-II Pareto solutions before live deployment. From each offline optimisation run, the knee-point chromosome $\mathbf{c}^*$ is decomposed into supervised training pairs:

$$\mathcal{D}_{\mathrm{BC}} = \left\{ (\mathbf{s}_j,\ a_j^*) \;\middle|\; a_j^* = x_j(\mathbf{c}^*),\ j = 1, \ldots, P \right\} \tag{56}$$

where $\mathbf{s}_j$ is the state vector at the moment step g_j would be placed, and a_j^* is the NSGA-II optimal action for that step. The network is trained on $\mathcal{D}_{\mathrm{BC}}$ by minimising cross-entropy loss:

$$\mathcal{L}_{\mathrm{BC}}(\boldsymbol{\theta}) = -\frac{1}{|\mathcal{D}_{\mathrm{BC}}|} \sum_{(\mathbf{s},a^*)\in\mathcal{D}_{\mathrm{BC}}} \log \frac{\exp Q(\mathbf{s}, a^*;\, \boldsymbol{\theta})}{\sum_{a\in\mathcal{A}} \exp Q(\mathbf{s}, a;\, \boldsymbol{\theta})} \tag{57}$$

This initialises the agent's policy close to NSGA-II's optimal policy, eliminating the initial period of near-random decisions.

7.7 Offline Training Data Generation

The offline training dataset $\mathcal{D}_{\mathrm{BC}}$ is constructed by running TOF-MMDE-NSGA-II on N_{batches} synthetic batches, each representing a plausible historical system state. For each batch $b \in \{1, \ldots, N_{\mathrm{batches}}\}$, the fog state is sampled as follows:

$$\rho_k^{(b)} \sim \mathcal{U}(0.20,\ 0.75), \quad k = 1, \ldots, K \tag{58}$$

$$q_k^{(b)} \sim \mathcal{U}(0,\ Q_{\max}), \quad k = 1, \ldots, K \tag{59}$$

$$s_i^{(b)} \sim \mathcal{N}(\mu_s,\ \sigma_s^2), \quad \phi_i^{(b)} \sim \mathcal{U}(0,\ 2\pi) \tag{60}$$

where $\mu_s = 60/3.6\,\mathrm{m/s}$ and $\sigma_s = 15/3.6\,\mathrm{m/s}$ are the mean and standard deviation of vehicle speed. Task MI values for pebble steps are drawn from the empirical distribution of the application DAG:

$$\ell_j^{(b)} \sim p_{\mathrm{MI}}, \quad p_{\mathrm{MI}} = \{200, 500, 800, 1000, 1200, 1500, 1800\} \text{ with equal probability} \tag{61}$$

Each batch yields P_{batch} pebble steps. After running NSGA-II and selecting the knee point $\mathbf{c}^{*(b)}$, the batch contributes P_{batch} labelled training pairs. The full dataset is:

$$\mathcal{D}_{\mathrm{BC}} = \bigcup_{b=1}^{N_{\mathrm{batches}}} \left\{ (\mathbf{s}_j^{(b)},\ a_j^{*(b)}) \;\middle|\; j = 1, \ldots, P_{\mathrm{batch}} \right\} \tag{62}$$

with total size $|\mathcal{D}_{\mathrm{BC}}| = N_{\mathrm{batches}} \cdot P_{\mathrm{batch}} = 10^5$ labelled examples at default settings.

7.8 Offline-to-Online Transition

Pre-training terminates when either the behavioral cloning loss falls below a convergence threshold ϵ_{BC} or the maximum number of epochs E_{BC} is reached:

$$\text{stop pre-training when: } \mathcal{L}_{\mathrm{BC}}(\boldsymbol{\theta}) < \epsilon_{\mathrm{BC}} \quad \text{or} \quad e = E_{\mathrm{BC}} \tag{63}$$

The weights at termination define the initial online policy:

$$\boldsymbol{\theta}_0 = \arg\min_{e \le E_{\mathrm{BC}}} \mathcal{L}_{\mathrm{BC}}(\boldsymbol{\theta}^{(e)}) \tag{64}$$

Both the online network and the target network are initialised to $\boldsymbol{\theta}_0$ before live simulation begins:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta}_0, \quad \boldsymbol{\theta}^- \leftarrow \boldsymbol{\theta}_0 \tag{65}$$

From this point, the agent operates under the ε-greedy policy of Eq. (45) and updates its weights via TD learning as each task step completes. The exploration rate ε is reset to ε_0 at the start of online deployment, ensuring continued exploration despite the informed initialisation.

8 Proactive Mobility Management

8.1 Zone Exit Time Estimation

The proactive handoff mechanism operates by comparing the estimated execution time of a step against the time available before the vehicle exits the serving fog node's coverage zone.

Definition 8.1 (Closing Speed)**.** The closing speed $v_{\mathrm{close}}(v_i, f_k, t)$ of vehicle v_i with respect to fog node f_k at time t is the radial component of the vehicle's velocity directed outward from the fog centre:

$$v_{\mathrm{close}} = \boldsymbol{u}_i(t) \cdot \hat{\boldsymbol{n}}_{ik}(t) \tag{66}$$

where $\hat{\boldsymbol{n}}_{ik}(t) = -(\boldsymbol{p}_k - \boldsymbol{q}_i(t))/\|\boldsymbol{p}_k - \boldsymbol{q}_i(t)\|_2$ is the unit outward radial vector from the fog centre toward the vehicle.

Definition 8.2 (Zone Exit Time)**.** The *zone exit time* $T_{\mathrm{exit}}(v_i, f_k, t)$ is the time in seconds until vehicle v_i crosses the boundary of fog node f_k's coverage zone, under linear extrapolation of current velocity:

$$T_{\mathrm{exit}}(v_i, f_k, t) = \begin{cases} \dfrac{R_k - \|\boldsymbol{q}_i(t) - \boldsymbol{p}_k\|_2}{v_{\mathrm{close}}} & \text{if } v_{\mathrm{close}} > 0 \\ +\infty & \text{if } v_{\mathrm{close}} \le 0 \end{cases} \tag{67}$$

$v_{\mathrm{close}} \le 0$ means the vehicle is moving toward or parallel to the fog centre, so no imminent exit is predicted.

8.2 Handoff Mode Selection

For each pebble step g_j assigned by the DQN agent to fog node f_k, the mobility management layer computes both T_{exit} and the effective execution time $T_{\mathrm{exec}}(g_j, f_k, t)$ via Eq. (15). The handoff mode is then selected as:

$$\mathrm{mode}(g_j, f_k, t) = \begin{cases} \textsc{Direct} & \text{if } T_{\mathrm{exec}} < T_{\mathrm{exit}} \\ \textsc{Proactive} & \text{if } T_{\mathrm{exec}} \ge T_{\mathrm{exit}} \end{cases} \tag{68}$$

Direct mode: The step will complete before the vehicle exits. No migration action is taken.

Proactive mode: The step will not complete before the vehicle exits. The predicted destination fog node $f_{k'}$ is identified by projecting the vehicle's position T_{exit} seconds forward:

$$\boldsymbol{q}_i^{\text{pred}} = \boldsymbol{q}_i(t) + T_{\text{exit}} \cdot \boldsymbol{u}_i(t) \tag{69}$$

The destination node is:

$$f_{k'} = \arg \min_{f_k \in \mathcal{F} \setminus \{f_k\}} \left\| \boldsymbol{q}_i^{\text{pred}} - \boldsymbol{p}_k \right\|_2 \quad \text{s.t.} \quad \left\| \boldsymbol{q}_i^{\text{pred}} - \boldsymbol{p}_{k'} \right\|_2 \leq R_{k'} \tag{70}$$

A container pre-spin request is immediately issued to $f_{k'}$, reserving $\mu_{k'} \cdot T_{\text{exec}}(g_j, f_{k'}, t)$ compute time and d_j^{in} kilobytes of memory for the arriving step. The vehicle connects to $f_{k'}$ at exit time and the step continues without interruption.

8.3 Reactive Fallback: NTB/HTB System

When trajectory prediction fails (unexpected turn, GPS signal loss), the reactive fallback activates. Tasks in the Normal Task Buffer (NTB) are processed in arrival order. Upon detecting vehicle disconnection, in-flight steps are moved to the Handoff Task Buffer (HTB):

$$\text{HTB} \leftarrow \text{HTB} \cup \{g_j \mid g_j \in \text{NTB},\ v_i(g_j) \text{ disconnected}\} \tag{71}$$

The HTB is assigned strict priority:

$$\text{scheduled CPU fraction for HTB} = 1.0 \quad \text{while } |\text{HTB}| > 0 \tag{72}$$

Completed results are held in a Central Tracker indexed by vehicle ID. Upon reconnection of vehicle v_i to any fog node f_k:

$$\text{deliver all results } \in \text{Tracker}[v_i] \text{ to } v_i \text{ via } f_k \tag{73}$$

This eliminates task re-submission and the associated retransmission cost.

8.4 Step-Level Assessment for DAG Tasks

Prior work assesses T_{exit} once per task [5]. This framework evaluates Eq. (67) independently for every DAG step g_j, against the specific fog node x_j assigned by the DQN agent:

$$\forall g_j \in \Pi : \quad \text{mode}(g_j) = \text{mode}(g_j, x_j(\mathbf{c}^*), t_j) \tag{74}$$

where t_j is the time at which step g_j begins transmission. Since different steps are assigned to different fog nodes, and the vehicle position evolves as each step executes, this per-step assessment produces more accurate migration decisions than task-level assessment.

9 Computational Complexity Analysis

9.1 Offline Phase: TOF-MMDE-NSGA-II

The dominant cost in the offline phase is NSGA-II. Let n be the population size, G the number of generations, P the number of pebble steps per batch, and K the number of fog nodes. The cost per generation is:

$$\mathcal{O}_{\text{gen}} = O(n^2) + O(n \cdot P \cdot K) \tag{75}$$

The $O(n^2)$ term accounts for non-dominated sorting, which compares every pair of solutions [10]. The $O(n \cdot P \cdot K)$ term accounts for objective evaluation: for each of the n chromosomes, all P genes are evaluated against all K destinations. Total offline cost across G generations and N_{batches} batches is:

$$\mathcal{O}_{\text{offline}} = N_{\text{batches}} \cdot G \cdot \big(O(n^2) + O(n \cdot P \cdot K)\big) \tag{76}$$

At default settings ($N_{\text{batches}} = 10^3$, $G = 200$, $n = 100$, $P = 100$, $K = 4$), the offline phase runs once before deployment and its cost is amortised over the full simulation lifetime.

9.2 Online Phase: DQN Inference

At runtime, the agent makes one placement decision per pebble step via a single forward pass through the Q-network. The cost of one inference is:

$$\mathcal{O}_{\text{infer}} = O(|\mathbf{s}| \cdot h_1 + h_1 \cdot h_2 + h_2 \cdot |\mathcal{A}|) \tag{77}$$

With $|\mathbf{s}| = 11$, $h_1 = 256$, $h_2 = 128$, $|\mathcal{A}| = 5$:

$$\mathcal{O}_{\text{infer}} = O(11 \cdot 256 + 256 \cdot 128 + 128 \cdot 5) = O(35{,}776) \tag{78}$$

This is a fixed constant independent of the number of tasks, fog nodes, or vehicles. On modern hardware a single inference executes in under 1 ms, satisfying the real-time constraint imposed by the 200 ms end-to-end deadline. This is the key computational advantage over running NSGA-II at inference time, which would require $O(n \cdot P \cdot K \cdot G)$ operations per decision.

9.3 Mobility Prediction

Computing T_{exit} via Eq. (67) requires one dot product and one division: $O(2)$ per step. The proactive destination search Eq. (70) evaluates $K - 1$ distance comparisons: $O(K)$. The total per-step mobility cost is $O(K)$, which at $K = 4$ is negligible.

9.4 Summary

Component	When	Complexity per call
TOF-Broker (EC classification)	per step	$O(1)$
NSGA-II per generation	offline	$O(n^2 + n \cdot P \cdot K)$
Knee-point selection	offline	$O(n)$
DQN inference (Agent 1)	per step	$O(36{,}288)$ constant
DQN weight update	per step	$O(B \cdot h_1 \cdot h_2)$
T_{exit} computation	per step	$O(K)$
HTB queue operation	on event	$O(1)$

10 End-to-End Processing Pipeline

The complete processing pipeline for a single DAG task τ submitted by vehicle v_i is described by Algorithm 2. It integrates all five algorithmic components into a unified sequential procedure.

Algorithm 2 PCNME End-to-End Task Processing

Input: DAG task τ, fog state $\{\rho_k, q_k\}$, vehicle state σ_i, pre-trained agent $Q(\cdot; \boldsymbol{\theta})$
Output: completed task with latency L_τ, energy E_τ, deadline verdict
for each offloadable step $g_j \in G$ **do**
  Compute $\mathrm{EC}(g_j)$ via Eq. (27)
  if $\mathrm{EC}(g_j) \geq \theta$ **then**
    Route g_j to $\mathcal{C}$ directly ▷ Boulder
    Execute and record L_j, E_j; **continue**
  end if
  if $Q(t) > Q_{\max}$ **then**
    Trigger Aggregator; bundle overflow pebbles to $\mathcal{C}$
  end if
  Build state vector $\mathbf{s}_j$ via Eq. (41)
  $a_j \leftarrow \varepsilon$-greedy$(Q(\mathbf{s}_j; \boldsymbol{\theta}))$ ▷ Agent placement decision
  $x_j \leftarrow \mathcal{X}[a_j]$ ▷ Map action to destination
  Compute $T_{\text{exit}}(v_i, x_j, t)$ via Eq. (67)
  Compute $T_{\text{exec}}(g_j, x_j, t)$ via Eq. (15)
  mode $\leftarrow$ select_mode via Eq. (68)
  if mode = PROACTIVE **then**
    Identify $f_{k'}$ via Eq. (70)
    Pre-spin container at $f_{k'}$ with resource reservation
    Redirect g_j to $f_{k'}$; update $x_j \leftarrow f_{k'}$
  end if
  Transmit g_j to x_j; record T_{access}
  Execute g_j on x_j; record T_{exec}
  $L_j \leftarrow T_{\text{access}} + T_{\text{exec}}$; $E_j \leftarrow$ Eq. (20)
  $r_j \leftarrow R(g_j, a_j)$ via Eq. (47)
  Store transition (s_j, a_j, r_j, s_{j+1}) in $\mathcal{B}$
  Sample $\mathcal{B}' \subset \mathcal{B}$; update $\boldsymbol{\theta}$ via Eq. (53)
end for
$L_\tau \leftarrow \sum_{g_j \in \Pi} L_j$; $E_\tau \leftarrow \sum_j E_j$
Report: L_τ, E_τ, $L_\tau \leq D_\tau$

11 Evaluation Metrics

The following metrics are used to evaluate the framework, each derived from the formal quantities defined above.

$$\text{Task Feasibility Rate} = \frac{1}{|\mathcal{T}|} \sum_{\tau \in \mathcal{T}} \mathbf{1}[L_\tau \leq D_\tau] \tag{79}$$

$$\text{Average Latency} = \frac{1}{|\mathcal{T}|} \sum_{\tau \in \mathcal{T}} L_\tau \tag{80}$$

$$\text{Average Energy} = \frac{1}{|\mathcal{T}|} \sum_{\tau \in \mathcal{T}} E_\tau \tag{81}$$

$$\text{Handoff Success Rate} = \frac{\text{handoffs with no re-submission}}{\text{total handoff events}} \tag{82}$$

$$\text{Pareto Hypervolume} = \lambda\left(\bigcup_{\mathbf{c} \in \mathcal{P}^*} \left\{ \boldsymbol{y} \mid \boldsymbol{f}(\mathbf{c}) \preceq \boldsymbol{y} \preceq \boldsymbol{f}^{\text{ref}} \right\} \right) \tag{83}$$

where $\lambda(\cdot)$ in (83) denotes Lebesgue measure and $\boldsymbol{f}^{\text{ref}}$ is a reference point dominated by all Pareto solutions. Hypervolume is a quality indicator for the offline Pareto front that captures both proximity to the true front and diversity of solutions.

References

[1] V. Jafari and M. H. Rezvani, "Joint optimization of energy consumption and time delay in IoT–fog–cloud computing environments using NSGA-II metaheuristic algorithm," *J. Ambient Intell. Humanized Comput.*, vol. 14, pp. 1675–1698, 2023.

[2] N. Nouri and M. H. Rezvani, "A multi-objective approach for optimizing IoT applications offloading in fog–cloud environments with NSGA-II," *J. Supercomput.*, 2024.

[3] Y. Li et al., "Energy-efficient resource management in microservices-based fog and edge computing: state-of-the-art and future directions," *ACM Comput. Surv.*, 2025.

[4] L. Zhu et al., "Dynamic task offloading scheme for edge computing via meta-reinforcement learning," *J. Manuf. Syst.*, 2025.

[5] S. Zaidi et al., "Task offloading optimization using PSO in fog computing for the internet of drones," *Drones*, vol. 8, no. 11, p. 696, 2024.

[6] M. P. Toopchinezhad and M. Ahmadi, "Deep reinforcement learning for delay-optimized task offloading in vehicular fog computing," *arXiv:2410.03472*, 2024.

[7] M. Zhu, J. Liu, and H. Wang, "A collaborative offloading task framework for IoT fog computing," in *Proc. Int. Conf. Fog Mobile Edge Comput.*, 2023.

[8] K. Gasmi and T. Ezzedine, "Reinforcement learning based task offloading of IoT applications in fog computing," *Cluster Comput.*, 2024.

[9] L. Zhu et al., "A vehicular edge computing offloading and task caching solution based on spatiotemporal prediction," *Future Gener. Comput. Syst.*, vol. 166, p. 107679, 2025.

[10] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Trans. Evol. Comput.*, vol. 6, no. 2, pp. 182–197, Apr. 2002.

[11] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, Feb. 2015.

[12] D. P. Kingma and J. Ba, "Adam: a method for stochastic optimization," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2015.

[13] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, Cambridge, MA, 2018.

[14] P. J. Huber, "Robust estimation of a location parameter," *Ann. Math. Stat.*, vol. 35, no. 1, pp. 73–101, 1964.

[15] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: the communication perspective," *IEEE Commun. Surv. Tuts.*, vol. 19, no. 4, pp. 2322–2358, 2017.

[16] I. Mashal et al., "Multiobjective offloading optimization in fog computing using deep reinforcement learning," *J. Comput. Netw. Commun.*, 2024.